

Integration of MUL Instruction into PicoRV32

Vinay Kusumanchi

Execution Flow Implemented:

- 1 Check instruction valid
- 2 Decode instruction
- 3 Execute in ALU
- 4 Write back to register file

Design Decision:

- MUL integrated directly into ALU
- Not implemented through PCPI

RISC-V MUL Encoding

R-Type Instruction

- Opcode = 0110011
- funct3 = 000
- funct7 = 0000001

Binary Format:

0000001 *rs2 rs1* 000 *rd* 0110011

1. Control Signal Declaration

```
reg instr_mul;
```

- New control signal added
- Asserted when MUL detected

2. Decode Logic

```
instr_mul <= is_alu_reg_reg &&  
             mem_rdata_q[14:12] == 3'b000 &&  
             mem_rdata_q[31:25] == 7'b0000001;
```

- Checks R-type instruction
- Verifies funct3 and funct7
- Differentiates MUL from ADD

3. Datapath Addition

```
reg [31:0] alu_mul;
```

- Stores multiplication result
- Supports TWO_CYCLE_ALU mode

4. Execution Stage

```
alu_mul <= reg_op1 * reg_op2;
```

- Performs multiplication
- Lower 32 bits written to rd

5. ALU Integration

```
instr_mul:
    alu_out = TWO_CYCLE_ALU ? alu_mul :
        (reg_op1 * reg_op2);
```

- Integrated into ALU case statement
- Compatible with existing ALU operations

Complete MUL Signal Flow

- 1 Fetch instruction from memory
- 2 Check instruction valid
- 3 Decode $\rightarrow$ assert `instr_mul`
- 4 Read `rs1` and `rs2`
- 5 Execute multiplication
- 6 Write result back to `rd`

Key Points

- Integrated into core datapath
- No modification to PCPI module
- Maintains PicoRV32 timing model
- Only MUL implemented